```
while connection.Process(1):
        pass
```

Save the code in the file *base.py* and run it:

```
$ chmod a+x base.py
$ ./base.py
```

Now run a Pidgin instance and this client at the same time. Use two different GTalk accounts to log in with Pidgin and this client. After running this script, in Pidgin, hover the mouse pointer over the script user (the user under whose account the script authenticated itself). Check the status—"LFY-client" — that has been set by the script. Try sending messages to the script user account— of course, it will not respond, because this is still a very basic client and we haven't specified what to do when some message arrives.

Now, let's find out how to look into the XML stream. Switch the debugging on: change the *connection = xmpp.Client(server,debug=[])* line to *connection = xmpp.Client(server)* by removing the *debug=[]* Parameter. Now when you run this script, you can see the XML stream, as shown in Figure 1.

Figure 1: Debug information (XML stream) in the terminal window

Writing a GTalk bot

You might have used or encountered GTalk bots that auto-respond to messages—for example, Google's own transliteration bots. If we add a transliteration bot as a buddy and send a phonetic word to it, it will send back a Unicode string transliterated to the corresponding Indic language. It isn't hard to develop something like that if you have a back-end application that can be used for transliteration. Now, we will modify the current *base.py* by adding a message handler. When a message is received, this handler will reply with the message "Welcome to my first GTalk Bot :)".

```
#!/usr/bin/env python

import xmpp

user="username@gmail.com"
password="password"
server="gmail.com"

def message_handler(connect_object, message_node):
        message = "Welcome to my first Gtalk Bot :)"
        connect_object.send( xmpp.Message( message_node.getFrom()
,message))

jid = xmpp.JID(user)
connection = xmpp.Client(server)
connection.connect()
result = connection.auth(jid.getNode(), password, "LFY-client")
connection.RegisterHandler('message', message_handler)

connection.sendInitPresence()
```

```
while connection.Process(1):
        pass
```

Here, *connection.RegisterHandler('message',messag e_handler)* is used to specify that the *message_handler()* function must be called when a message stanza is received. The two arguments passed to the function are a connection object, and the message stanza node. By using the attribute function *getFrom()*, we obtain the JID of the user who sent the message stanza, and we send our reply to that user. To obtain the text of the received message, use the function *message_ node.getBody()*.

Remote-control shell bot

You have seen how to write a very simple bot in a few lines of code. Now let's play with another idea: remotely controlling computers via an XMPP bot. Normally, we use SSH (Secure Shell) for remote administration—it gives us a shell at the remote machine, so we can execute commands on that computer. Can we do something similar with an XMPP bot? Yes! Let's modify our simple bot to act as a remote-controlled shell bot. Replace the simple bot's *message_handler()* function with this new one:

```
def message_handler(connect_object, message_node):
        command = str(message_node.getBody())

        process = subprocess.Popen(command,shell=True,stdout=subprocess.
PIPE, stderr=subprocess.PIPE)
        message = process.stdout.read()
        if message=="":
            message=process.stderr.read()

        connect_object.send( xmpp.Message( message_node.getFrom()
,message))
```

Note: You will need to add *import subprocess* at the top of the program file, since the module is now used in the *message_handler()* function.

In this message handler, we retrieve the text of a message received from a sender, and run it as a command